IE 303 Modeling and Methods in Optimization

The Computing Project

Bora Çetin ✉
Emir Doğan ✉
Yaren Nehir Çoban ✉

Bilkent University
Department of Industrial Engineering
Fall 2024-2025

Introduction and Motivation

This assignment will count as your project grade. (10% overall) Even though modeling optimization projects can be tricky, solving the problem with a solver is an issue on its own for many applications, but especially to those involving integer variables. The goal of this computing project is to make students get used to state-of-art solvers, in particular, to `gurobipy`.

This assignment can be completed in groups of 2. Write a *TYPED* report including your answers, methodology, and if any, assumptions and please convert it to a *.pdf* file. Do not include any code screenshots in your *.pdf* file. Instead, place your written material and `gurobipy` python codes in a *.zip* file and submit it with name format

NameSurname_NameSurname_303Project.zip

Then, upload your solutions to Moodle. One submission per group is enough. Your project report should include the names of the group members.

The deadline to this assignment is **09.12.2024, 23:59**. Your *.zip* file should include one *.pdf* file (your typed report) and four python scripts including your models in `gurobipy` for each question, hence in total there should be 5 files. Please prepare your reports *on your own*, and with your *own* sentences. Good luck!

Grading Scheme

Late submission grades are calculated according to the formula below. Let $\mathcal{G}$ be your original grade. Let $x \in \mathbb{R}$ be the amount of late submission in hours. Then your final grade will be calculated according to

$$\mathcal{G}_{\textbf{new}}(x) = \begin{cases} \mathcal{G}e^{-\frac{x}{40\ln 8}} & \text{if } 0 \leq x \leq 24 \\ 0 & \text{if } x > 24 \end{cases}$$

So if you are about 24 hours late, then you lose approximately about 20% of your project grade. Submissions that are more than 24 hours late will not be accepted and will be assigned a grade of 0.

Question 0 (Install `gurobipy`)

(10pts) Install `gurobipy` (Gurobi Optimization Python API) to your computers. In addition to the questions below, you are getting 10% of your grade from formatting and gurobipy installation.

Question 1 (Warm-Up)

(10pts) Consider the following IP.

$$\begin{aligned} \textbf{minimize} \quad & 15x_1 + 10x_2 \\ \textbf{subject to} \quad & 4x_1 + x_2 \geq 6 \\ & x_1 + 2x_2 \geq 3 \\ & x_1, x_2 \in \mathbb{Z}_+ \end{aligned}$$

a) Solve it using `gurobipy`. Report the optimum solution.

b) Now, solve the LP relaxation of this problem. How far are you from the integer optimum solution? If your solution to this part is not an integer solution, add a valid inequality that is also a cutting plane to this problem, cutting away the LP relaxation optimum solution. You can use any method to find such a cut, but in your report you should show that your inequality is indeed *valid*.

Question 2 (Generating Cutting Planes)

(30pts) Consider the following integer programming problem. Let the problem parameters be $\mathbf{c}, \mathbf{a} \in \mathbb{R}^n_+$ and $b \in \mathbb{R}_+$.

$$\begin{aligned} \textbf{maximize} \quad & \mathbf{c}^\intercal \mathbf{x} \\ \textbf{subject to} \quad & \mathbf{a}^\intercal \mathbf{x} \leq b \\ & \mathbf{x} \in \mathbb{B}^n \end{aligned}$$

a) We have seen this problem in the lectures. What is it?

b) Write a python code that does the following. It takes vectors $\mathbf{c}$, $\mathbf{a}$ and b as inputs and solves the corresponding optimization problem, then returns the optimum vector $\mathbf{x}^*$ as well as the optimum value z^*. Decide parameters on your own for a problem with $n = 5$ and solve it using your code. Clearly indicate your vectors and parameters in the report, as well as the optimum solution.

c) Write a python code that does the following.

1. It takes vectors $\mathbf{c}$, $\mathbf{a}$ and b as inputs, solve the *LP Relaxation* of the problem above.
2. If the solution vector is not integer feasible, find and add a cover inequality to the LP relaxation, which cuts the LP optimum.
3. If not integer, extend this cover inequality (if possible) and resolve the problem.
4. If the integer solution is still not obtained, go to Step 2. and repeat this procedure.

5. If one gets an integer solution or there are no cover inequalities left to add, stop.
6. Report the optimum vector $\mathbf{x}^*$ and the optimum value z^*, as well as the number of cover inequalities added.

Your code should work for any $\mathbf{c}, \mathbf{a} \in \mathbb{R}^n_+$ and $b \in \mathbb{R}_+$ provided that the problem is feasible. Using your code, solve the following example.

$$\mathbf{c} = \begin{bmatrix} 17 & 35 & 19 & 67 & 3 & 8 & 42 \end{bmatrix}$$

$$\mathbf{a} = \begin{bmatrix} 5 & 11 & 8 & 18 & 21 & 24 & 18 \end{bmatrix}$$

$$b = 45$$

Report your final formulation for this instance (together with all cover inequalities you added until the optimum)

Question 3 (Mission: 007 Possible)

(25pts) Imagine there are 7 secret agents, each in a different safe house, communicating with each other via secret channels. These channels are currently two-way (they can send and receive messages to each other). Your job is to redesign all of these communication channels so that each channel is one-way (messages only flow in one direction) to reduce the risk of detection.

Any agent can communicate with an intermediary agent to send the message, which basically ensures that no communication is lost. Can you set up (minimum number of) one-way channels in a way that still guarantees all agents communicating with each other? One can check seven towns and one-way roads. Use integer programming to solve this problem. Report your formulation and your final network.

Question 4 (The Magic Squares)

(25pts) A *magic square* is a grid of distinct positive integers arranged such that the sum of the numbers in each row, each column, and both diagonals is the same. The positive integers must appear exactly once in the grid.

For example, given a 3×3 grid, and the goal is to create a magic square with integers between 1 and 9 such that the sum of each row, column, and diagonal is equal to 15.

a) Develop an integer programming model for creating an $n \times n$ magic square. Clearly describe your model, constraints and variables. Explain how they ensure the properties of a magic square.

b) Using your model from part a), specify how it would apply to an 8×8 grid. The grid should contain each integer from 1 to 64 exactly once, with each row, column, and diagonal summing to the magic constant for an 8×8 magic square. Clearly outline the model setup and parameter values specifically for the 8×8 case.

c) Solve the model you have described in part b) with `gurobipy`. What is the resulting 8×8 magic square?

d) To explore the computational complexity of solving a magic square model, implement and solve the model for each grid size n from 3×3 up to 10×10, record the runtime of each model. Plot the *runtime* of each grid size with respect to the square size.